

mwm: A Minimal Runtime-Configurable X11 Window Manager

Mohammed Bakr

September 9, 2026

1 Overview

mwm is a minimal tiling and floating window manager for X11, written in C using Xlib. It supports both tiling and floating window arrangements, multiple layout modes, workspaces, and per-application rules. Unlike an earlier version of this project, configuration is read from a plain text file at startup rather than compiled into the binary, allowing keybinds, colors, and behavior to be changed without recompilation.

2 Design Goals

- Small, readable codebase with no external dependencies beyond Xlib.
- Runtime configuration through a simple key-value file format.
- Support for both tiling and floating window management, selectable per window and per workspace.
- Predictable, minimal feature set in the spirit of suckless-style window managers.

3 Architecture

The project is organized into five translation units, each responsible for one area of functionality.

- **mwm.c**: X server connection, event loop, and window lifecycle management (mapping, unmapping, destruction).
- **config.c**: Parses the runtime configuration file and populates the global configuration structure, including keybinds and window rules.
- **client.c**: Maintains the linked list of managed windows (clients), including creation, destruction, lookup, and focus handling.
- **layout.c**: Computes window geometry for the tiling and monocle layouts. Floating windows retain their own geometry and are left untouched by this module.
- **actions.c**: Implements the behavior triggered by each keybind, such as spawning programs, changing layouts, and switching workspaces.

3.1 Core Data Structures

The `Client` structure represents a single managed window. It stores the window's current geometry, its stored floating geometry (so a window can return to its previous size and position after being tiled), its workspace, and its floating and fullscreen state.

The `Config` structure holds all values loaded from the configuration file: the modifier key, border appearance, tiling ratio, gap size, the list of keybinds, and the list of per-application rules.

The `WM` structure is the single global instance holding the X display connection, the root window, screen dimensions, the client list, the currently focused client, and the active layout for each workspace.

4 Configuration File

The configuration file is located at `~/.config/mwm/mwmrc`. It uses a simple line-based format:

```
key = value
bind = mod+key, action, arg
rule = ClassName, workspace=N, floating=0 or 1
```

Lines beginning with `#` are treated as comments and ignored by the parser. If no configuration file is found, built-in defaults are used instead.

4.1 General Settings

- `mod`: the modifier key used for keybinds, either `super` or `alt`.
- `border_width`: window border thickness in pixels.
- `border_focus` and `border_unfocus`: border colors in hexadecimal, used for the focused and unfocused windows.
- `master_ratio`: fraction of the screen width given to the master area in tiling mode.
- `gap`: spacing in pixels between tiled windows and the screen edge.

4.2 Keybinds

Each `bind` line associates a key combination with an action and an optional argument. The key combination is written as a sequence of modifiers and a key name joined with `+`, for example `mod+shift+Return`. Supported actions include spawning a program, killing the focused window, moving focus, toggling floating or fullscreen state, changing the layout, switching or moving windows between workspaces, resizing the master area, reloading the configuration, and quitting.

4.3 Rules

A `rule` line assigns a workspace or floating state to windows of a given class name, applied automatically when the window is first managed.

5 Event Handling

mwm operates on a single-threaded event loop built around `XNextEvent`. The events handled are:

- `MapRequest`: a new window is being mapped and is brought under management.

- **DestroyNotify** and **UnmapNotify**: a managed window is removed from the client list and the layout is recomputed.
- **KeyPress**: the pressed key combination is matched against the configured keybinds, and the corresponding action is invoked.
- **EnterNotify**: used to implement focus-follows-mouse behavior.

6 Layout Engine

Three layout modes are supported, selectable per workspace:

- **Tile**: the first window occupies a master area on the left, sized according to `master_ratio`. Remaining windows are stacked vertically on the right.
- **Monocle**: all windows occupy the full available screen area, stacked on top of each other.
- **Float**: no automatic geometry is applied; windows keep whatever position and size they were given.

Fullscreen windows bypass the active layout and occupy the entire screen regardless of mode.

7 Building and Installation

The project is built with a plain Makefile and requires only the Xlib development headers.

```
make
sudo make install
```

Installation places the binary at `/usr/local/bin/mwm` and copies a default configuration file to the user's home directory if one does not already exist.

8 Testing

Because running an unfinished window manager directly in a login session carries risk, mwm can be tested inside Xephyr, a nested X server that runs as a window within an existing session.

```
Xephyr -screen 1280x800 :1 &
DISPLAY=:1 ./mwm
```

This allows the window manager to be started, interacted with, and restarted freely without affecting the outer session.

9 Future Work

Planned extensions include mouse-based dragging and resizing of floating windows, an optional status bar, and multi-monitor support.